Phase 3 — Semantic Deduplication (Cross-Source Same-Story Detection)

Thesis priority: #2 (cross-source duplicate detection) from §5.5 Future Works. **Date:** 2026-10-06 (FYP 2, Week 1)
Status: implemented and evaluated against the production corpus; default threshold 0.85.

---

1. Problem statement (from FYP 1 evaluation, thesis §5.4)

Token/Jaccard similarity can fail when different publishers describe the same incident with different wording, causing repeat notifications.

Concrete FYP 1 weakness (`src/core/services.py`): the cross-source story matcher compares token sets (Jaccard) over title / body / AI summary. When two publishers use different words for the same incident — synonyms, different sentence structures, or a different language (English vs Bahasa Malaysia, extremely common in Sarawak sources) — token overlap collapses and both versions are delivered as "new" stories.

2. Changes made

2.1 Design

The FYP 2 vector table (`article_embeddings`, Phase 2) already stores one embedding per article over `title + ai_summary + category + location`. Same-story articles embed close together even when their wording differs, so the semantic check needs **no new infrastructure and no LLM calls** — it is pure SQL + in-memory cosine between already -stored vectors.

The semantic pass is an **addition to dedup layer 2**, not a replacement (AGENTS.md preserves three dedup layers):

1. unique-link check (unchanged),
2. cross-source story matching: Jaccard pass first (cheap, exact-wording duplicates), then the new cosine pass on pairs Jaccard cleared,
3. per-user `UserArticleDelivery` (unchanged).

Scope decision: the cosine pass runs at **delivery-time clustering** (`_cluster_ranked_articles_cross_source`), where articles have DB ids and (usually) vectors. Ingestion-time RSS-item dedup (`_deduplicate_items`) is left Jaccard-only: items are not yet in the DB, and embedding raw items would cost Ollama calls per item. Duplicate DB rows still get *clustered* before any user sees them, which is what prevents repeat notifications.

2.2 Config (`src/core/config.py`)

Variable	Default	Purpose
`CROSS_SOURCE_DEDUP_SEMANTIC_ENABLED`	`true`	Master switch; `false` = Jaccard-only (exact FYP 1 behavior).
`CROSS_SOURCE_DEDUP_SEMANTIC_COSINE_THRESHOLD`	`0.85`	Same-story floor, set from the labeled-pair eval below.

2.3 Code

- `src/ai/retriever.py`:
- `fetch_stored_vectors(article_ids)` — one batched SQL fetch of embedding rows; returns `None` on any failure so callers skip the semantic pass (graceful degradation, rule #8). pgvector text literals are coerced defensively (`_coerce_vector`) — the pooler returns vectors as `'[f,f,…]'` text without a type codec.
- `cosine_between(a, b)` — pure-python cosine.

`src/core/services.py` — `_cluster_ranked_articles_cross_source`: vectors for the batch are fetched once up front; after the Jaccard matcher clears a pair, cosine against each existing cluster representative ≥ threshold groups the article with `match_by="semantic"` (visible in `CROSS_SOURCE_DEDUP_DEBUG` logs). Articles without embedding rows are still clustered by Jaccard alone.

No schema change → no migration (change rule #2 satisfied trivially).

3. Evaluation (2026-10-06, production Supabase corpus)

`scripts/eval_semantic_dedup.py`; full output `benchmark/dedup_semantic_20261006.txt`, pair data `benchmark/dedup_pair_cosines.csv`.

3.1 Threshold evidence (249 labeled pairs, 45-day corpus, 1,739 embedded articles)

Pair set	n	min	p25	median	p75	max
`jaccard_caught` (Jaccard would flag)	40	0.755	0.870	0.927	0.965	1.000
`hard_positive` (same location, ≤48h apart, Jaccard clears)	149	0.570	0.675	0.707	0.774	0.957
`negative` (different location or >7d apart)	60	0.492	0.625	0.667	0.688	0.765

Reading: true same-story pairs form a high-cosine band (the `jaccard_caught` row and the top of the `hard_positive` range); negatives stay ≤ 0.77. **0.85 sits in the gap** — above every sampled negative and below the clear same-story cluster. It deliberately keeps the gray band (0.77–0.85) unmerged: those pairs are ambiguous, and an unmerged possible duplicate is a far cheaper error than a false merge that hides a distinct story.

3.2 Before/after simulation (articles from the last 14 days, production clustering code)

	Clusters
Jaccard only (FYP 1)	312
Jaccard + semantic	249
Merges made **only** by the semantic pass	71

Manual review of the 30 sampled semantic-only merges (~23% estimated false-merge rate): the true merges include **cross-lingual pairs Jaccard can never catch** — e.g. "Underground gas leak prompts shutdown at Kuala Baram oil palm site" ↔ "Gas bawah tanah bocor, ladang sawit ditutup sementara" (0.943), the Long Lellang helicopter-crash widow follow-ups (0.869), IPU/haze Malay↔English updates (0.856–0.864), utility repair updates (0.948–0.962), and the Datuk Song Swee Guan obituary cluster (0.953–0.977). Typical false merges are same-genre event roundups ("Sarawak Borneo Craft Festival targets RM3 mln in sales" ↔ "BCG 2.0 targets 30,000 visitors", 0.885).

Cost of a false merge is **soft**: clustering controls what a digest displays; merged members still contribute their source links to the cluster's "Sources:" line, so the reader can reach the suppressed article, but its title/summary are not shown.

4. Reproduction

```
python scripts/eval_semantic_dedup.py # pair histograms + before/after clustering sim
```

5. Threats to validity / risks

1. **Heuristic labels**: `hard_positive` pairs are labeled by location+time heuristics, not human annotation; the histogram (not the label) is the actual threshold evidence. A proper human-labeled set is Phase 5 evaluation work.
2. **False merges hide distinct stories** (see §3.2): ~23% at 0.85 by manual sample. Mitigations if this hurts in practice: raise the threshold (cross-lingual catches start dropping above ~0.89) or require an additional weak signal (shared distinctive token) alongside cosine.
3. **Fresh articles may lack embeddings** when Ollama was down at prefetch time — they silently get Jaccard-only treatment until embedded (observed: 7/1,746 unembedded right after a prefetch cycle).
4. **Batch composition matters**: the 312→249 figure is one 14-day global batch; per-user digest batches are smaller, so the delivered-item reduction will be smaller in absolute terms (though the same *rate* applies to every

digest).

5. Embedding quality still bounds detection: same-story pairs with very different framings can sit under 0.85 (floor of `jaccard_caught` is 0.755) — those remain for the Jaccard/body layer or slip through.